阿里云 × CLOUD NATIVE COMPUTING FOUNDATION

云原生技术公开课

第 08 讲

应用配置管理

新胜 阿里巴巴技术专家

1 需求来源

2 ConfigMap

3 Secret

4 ServiceAccount

5 Resource

6 SecurityContext

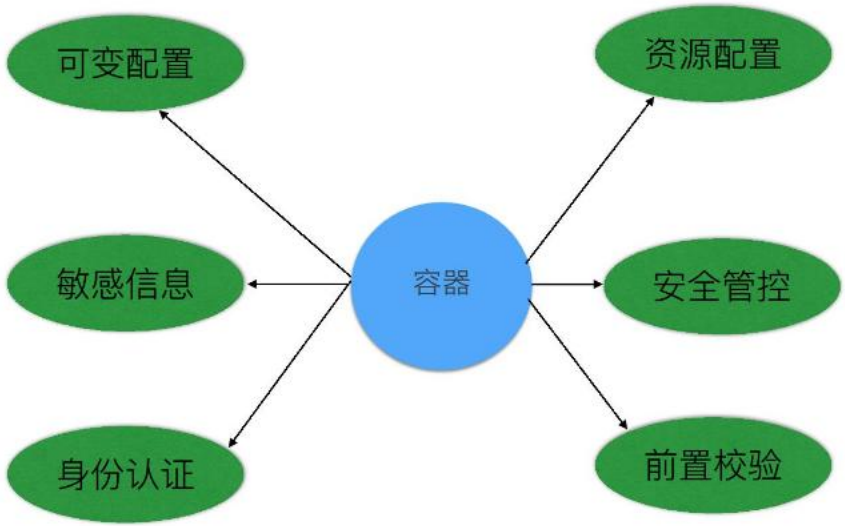
7 InitContainer

8 课后习题

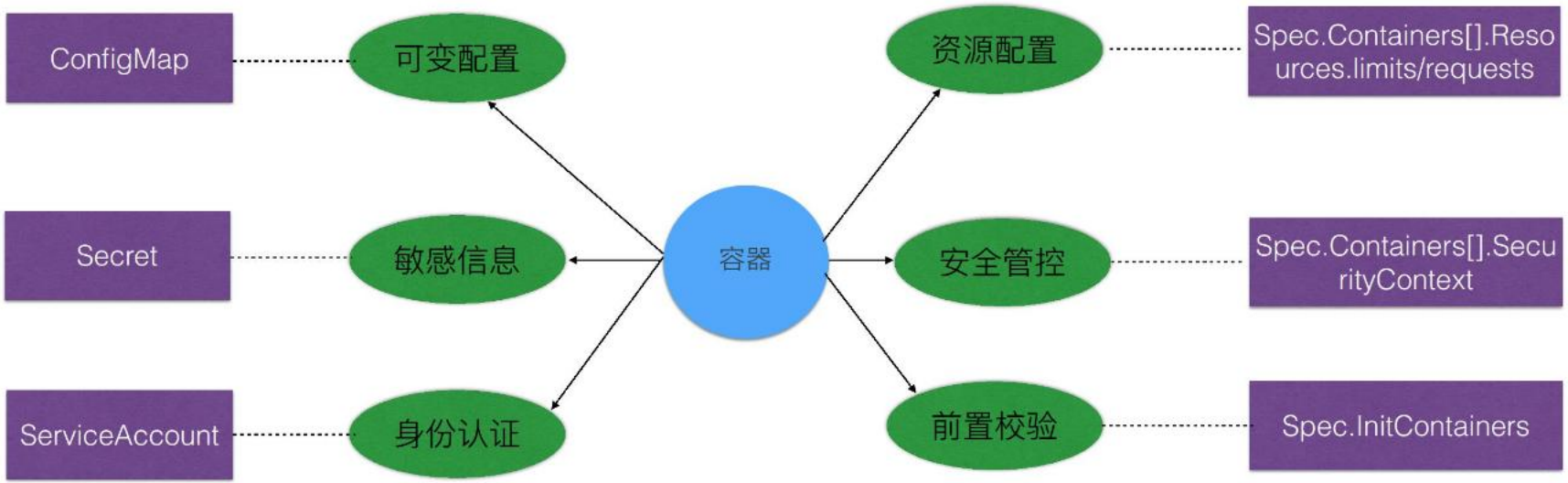
背景问题

除了依托容器镜像来定义运行的Container，Pod还需要解决如下问题

1. 不可变基础设施(容器)的可变配置
2. 敏感信息的存储和使用(如: 密码, token等)
3. 集群中Pod自我的身份认证
4. 容器运行资源的配置管理
5. 容器的运行安全管控
6. 容器启动前置条件校验等



Pod的配置管理


ConfigMap
可变配置
Secret
敏感信息
ServiceAccount
身份认证
容器
资源配置
安全管控
前置校验
Spec.Containers[].Resources.limits/requests
Spec.Containers[].SecurityContext
Spec.InitContainers

1 需求来源

2 ConfigMap

3 Secret

4 ServiceAccount

5 Resource

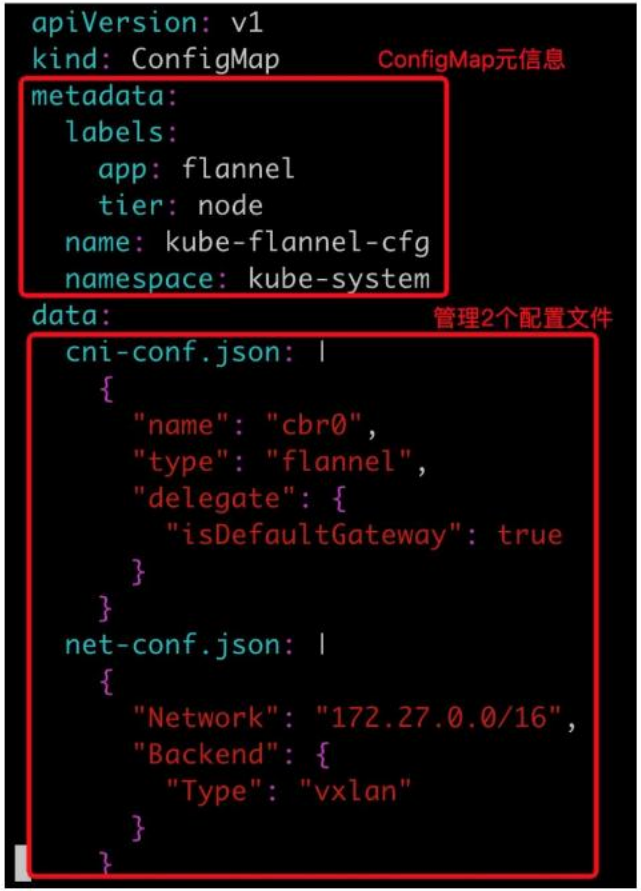
6 SecurityContext

7 InitContainer

8 课后习题

ConfigMap介绍

主要管理容器运行所需的配置文件，环境变量，命令行参数等可变配置。用于解耦容器镜像和可变配置，从而保障工作负载(Pod)的可移植性。




```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app: flannel
    tier: node
  name: kube-flannel-cfg
  namespace: kube-system
data:
  cni-conf.json: |
    {
      "name": "cbr0",
      "type": "flannel",
      "delegate": {
        "isDefaultGateway": true
      }
    }
  net-conf.json: |
    {
      "Network": "172.27.0.0/16",
      "Backend": {
        "Type": "vxlan"
      }
    }
```

ConfigMap创建

创建命令: kubectl create configmap [NAME] [DATA]

其中DATA:

- 指定文件或者目录
- 指定键值对

指定文件: kubectl create configmap kube-flannel-cfg **--from-file**=configure-pod-container/configmap/cni-conf.json -n kube-sytem

```
apiVersion: v1
kind: ConfigMap
metadata:
  labels:
    app: flannel
    tier: node
  name: kube-flannel-cfg
  namespace: kube-system
data:    文件名作为Key
  cni-conf.json: |        文件内容为value
    {
      "name": "cbr0",
      "type": "flannel",
      "delegate": {
        "isDefaultGateway": true
      }
    }
```

指定键值对: kubectl create configmap special-config **--from-literal**=special.how=very **--from-literal**=special.type=charm

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: special-config
  namespace: default
data:          输入的键值对
  special.how: very
  special.type: charm
```

ConfigMap使用

ConfigMap主要被Pod使用，一般用于挂载Pod用的配置文件，环境变量，命令行参数等。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cm-env-test
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "env" ]
      env:
        # 用special-config中的special.how定义环境变量
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: special.how
  restartPolicy: Never
```

用ConfigMap配置环境变量

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cm-cmd-test
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "echo $(SPECIAL_LEVEL_KEY)" ]
      env:
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: SPECIAL_LEVEL
  restartPolicy: Never
```

在命令行参数中使用ConfigMap定义的环境变量

用ConfigMap配置管控命令行参数

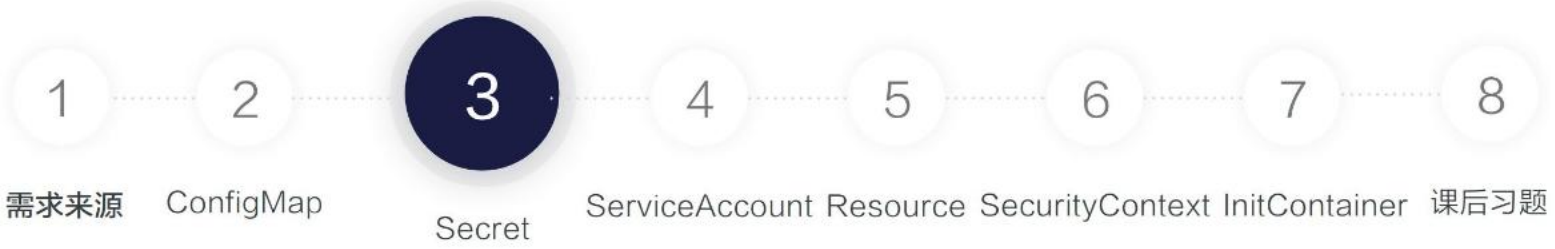
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cm-volume-test
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "ls /etc/config/" ]
      volumeMounts:
      - name: config-volume
        mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: special-config
  restartPolicy: Never
```

ConfigMap中指定的内容以文件形式挂载在容器中的/etc/config目录下

用ConfigMap挂载配置文件

ConfigMap使用注意点

1. ConfigMap文件大小限制: 1MB （*ETCD的要求）
2. Pod只能引用相同Namespace中的ConfigMap
3. Pod引用的ConfigMap不存在时，Pod无法创建成功。即Pod创建前需要先创建好ConfigMap。
4. 使用envFrom从ConfigMap来配置环境变量时，如果ConfigMap中的某些key被认为无效(比如key名称中带有数字)，该环境变量将不会注入容器，但是Pod可以正常创建。
5. 只有通过k8s api创建的pod才能使用ConfigMap，其他方式创建的pod(如manifest创建的static pod)不能使用ConfigMap


1
2
3
4
5
6
7
8
需求来源
ConfigMap
Secret
ServiceAccount
Resource
SecurityContext
InitContainer
课后习题

Secret介绍

Secret是在集群中用于存储密码，token等敏感信息用的资源对象。其中的敏感数据采用base-64编码保存，相比存储在明文的ConfigMap中更规范，更安全。



Secret主要有如下类型:

- type=Opaque
- type=kubernetes.io/service-account-token
- type=kubernetes.io/dockerconfigjson
- type=bootstrap.kubernetes.io/token
- …

Secret创建

Secret创建可以是用户自己创建，也有Secret是系统自动创建
- 比如: K8S为每个namespace的默认用户(default ServiceAccount)创建的Secret

手动创建命令: kubectl create secret generic [NAME] [DATA] [TYPE]
其中DATA: 可以指定文件/键值对
另外TYPE：默认为Opaque

指定文件: kubectl create secret generic myregistrykey **--from-file**=.dockerconfigjson=/root/.docker/config.json **--type=kubernetes.io/dockerconfigjson**

```
apiVersion: v1
kind: Secret
metadata:
  name: myregistrykey
  namespace: default
data:
  .dockerconfigjson: UmVhbGx5IHJlYWxseSB
xsbGxsbGxsbGxsbGxsbGxsbGx5eXl5eXl5eXl5
vb29vb25ubm5ubm5ubm5ubm5ubm5ubm5ubmdn
type: kubernetes.io/dockerconfigjson
```

指定键值对:kubectl create secret generic prod-db-secret **--from-literal**=username=produser
--from-literal=password=Y4nys7f11

```
apiVersion: v1
data:
  password: WTRueXM3ZjEx
  username: cHJvZHVzZXI=
kind: Secret
metadata:
  name: prod-db-secret
  namespace: default
type: Opaque
```

Secret使用

Secret主要被Pod使用，一般通过volume挂载到指定容器目录，供容器中业务使用。另外在需要访问私有镜像仓库时，也可以引用Secret来实现。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: mysecret
```

secret以文件形式挂载在/etc/foo下

指定要挂载的secret

Secert挂载在用户指定目录下

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-9f7575d75-mdfp6
  namespace: default
spec:
  containers:
  - image: nginx:1-alpine
    name: nginx
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: default-token-6fn7q
      readOnly: true
  serviceAccount: default
  serviceAccountName: default
  volumes:
  - name: default-token-6fn7q
    secret:
      defaultMode: 420
      secretName: default-token-6fn7q
```

固定挂载到容器中的/var/run/secrets/kubernetes.io/serviceaccount目录下

使用namespace默认的serviceaccount

挂载namespace默认的secret

ServiceAccount用的secret自动挂载在固定目录下(生成ca.crt和token两个文件)

使用私有镜像库

私有镜像仓库的信息可以通过Secret(kubernetes.io/dockerconfigjson)存储在集群中，Pod如果需要使用私有镜像仓库，可以通过如下两种方式来配置:

1. Pod.spec.imagePullSecrets来指定secret
2. ServiceAccount中设置imagePullSecrets，然后自动为使用该SA的Pod注入imagePullSecrets信息。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: private-reg
spec:
  containers:
  - name: private-reg-container
    image: <your-private-image>
  imagePullSecrets:
  - name: regcred
```

指定imagePullSecret

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: 2015-08-07T22:02:39Z
  name: default
  namespace: default
  selfLink: /api/v1/namespaces/default/serviceaccounts/default
  uid: 052fb0f4-3d50-11e5-b066-42010af0d7b6
secrets:
- name: default-token-uudge
imagePullSecrets:
- name: myregistrykey
```

自动为使用该ServiceAccount的Pod注入ImagePullSecrets信息

ServiceAccount设置，Pod创建时自动注入imagePullSecrets

Secret使用的注意点

1. Secret文件大小限制: 1MB
2. Secret虽然采用base-64编码，但是可以简单解码查看原始信息。因此机密信息采用Secret存储仍需要慎重考虑或者Secret访问者进行控制。对Secret加密有较强需求，可以考虑结合Kubernetes + Vault来解决敏感信息的加密和权限管理。
3. Secret最佳实践: 因为list/watch的一般处理将获取到namespace下所有secret，因此不建议采取list/watch方式获取Secret信息。而推荐使用GET来获取需要的Secret，从而减少更多Secret暴露的可能性。

阿里云 CLOUD NATIVE COMPUTING FOUNDATION



阿里云
CLOUD NATIVE COMPUTING FOUNDATION

ServiceAccount介绍

ServiceAccount主要用于解决Pod 在集群中的身份认证问题。其中认证使用的授权信息，则利用前面讲到Secret(type=kubernetes.io/service-account-token)进行管理.

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: 2019-06-03T08:32:59Z
  name: default
  namespace: default
  resourceVersion: "419220122"
  selfLink: /api/v1/namespaces/default/serviceaccounts/default
  uid: 320061cb-85da-11e9-909b-06b1d3b258c8
secrets:
- name: default-token-6fn7q
```

指定后台的Secret

SeviceAccount定义

```yaml
apiVersion: v1
data:
  ca.crt: LS0tLS1...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: default
    kubernetes.io/service-account.uid: 320061cb-85da-11e9-909b-06b1d3b258c8
  creationTimestamp: 2019-06-03T08:32:59Z
  name: default-token-6fn7q
  namespace: default
  resourceVersion: "419220120"
  selfLink: /api/v1/namespaces/default/secrets/default-token-6fn7q
  uid: 320182a6-85da-11e9-909b-06b1d3b258c8
type: kubernetes.io/service-account-token
```

用于校验服务端的证书信息，base-64编码

Pod的身份认证用的Token，base-64编码

关联的SeriviceAccount信息

Secret类型

ServiceAccount对应的Secret

*Secret由ServiceAccount Controller自动创建

阿里云 CLOUD NATIVE COMPUTING FOUNDATION

举例：Pod 里的应用访问它所属的 K8s 集群

实现原理:

1. Pod创建时Admission Controller会根据指定的ServiceAccount(默认为default)把对应的Secret挂载到容器中固定的目录下(/var/run/secrets/kubernetes.io/serviceaccount)。(*k8s自动实现)

2. 当 Pod 访问集群时，可以默认利用Secret其中的token文件来认证Pod的身份。(ca.crt用于校验服务端)(*参照右边代码)

3. 默认token的认证信息为:
   - Group: system:serviceaccounts:[namespace-name]
   - User: system:serviceaccount:[namespace-name]:[pod-name]

*同时Pod身份被认证合法后，其权限需要通过RBAC功能来配置。默认具备资源的GET权限。RBAC 的知识后续课程会详细介绍。

Pod访问集群默认实现代码说明

```
func InClusterConfig() (*Config, error) {
        const (
                tokenFile  = "/var/run/secrets/kubernetes.io/serviceaccount/token"
                rootCAFile = "/var/run/secrets/kubernetes.io/serviceaccount/ca.crt"
        )
        host, port := os.Getenv("KUBERNETES_SERVICE_HOST"), os.Getenv("KUBERNETES_SERVICE_PORT")
        if len(host) == 0 || len(port) == 0 {
                return nil, ErrNotInCluster
        }

        token, err := ioutil.ReadFile(tokenFile)
        if err != nil {
                return nil, err
        }

        tlsClientConfig := TLSClientConfig{}

        if _, err := certutil.NewPool(rootCAFile); err != nil {
                klog.Errorf("Expected to load root CA config from %s, but got err: %v", rootCAFile, err)
        } else {
                tlsClientConfig.CAFile = rootCAFile
        }

        return &Config{
                // TODO: switch to using cluster DNS.
                Host:            "https://" + net.JoinHostPort(host, port),
                TLSClientConfig: tlsClientConfig,
                BearerToken:     string(token),
                BearerTokenFile: tokenFile,
        }, nil
}
```

业务一般采用此方法生成ClientConfig

serviceaccount背后的secret文件

通过环境变量获取集群访问地址

ca.crt用于校验服务端是否可信

服务端将利用该token对Pod进行认证

https://github.com/kubernetes/client-go/blob/master/rest/config.go#L399:6

阿里云 CLOUD NATIVE COMPUTING FOUNDATION



100%
阿里云
CLOUD NATIVE COMPUTING FOUNDATION


Image loading...


Image loading...


Image loading...